

Sunrise School ERP — Simple Database Data-Flow Guide

A Visual Guide to How Data Moves Between Tables Across All Implemented Features

Database: PostgreSQL (93 Tables, 68 Live Tables)

Core Rule: TABLE → TABLE → TABLE

Scope: Verified Working Code Only

Target: Clear, Simple, Relational Flow

How to Read This Guide:

This document shows you **exactly how data moves across database tables** for every working feature in Sunrise ERP. For each feature, look at the visual chain (TABLE → TABLE), read what each table is responsible for, see the exact PK → FK connecting them, and read why the data moved.

1. Student ID vs Enrollment ID (The Core Distinction)

The Core Rule of Sunrise ERP:

- **Student ID** (`students.id` / `admission_no`): Identifies the student **permanently for life**.
- **Enrollment ID** (`enrolments.id` / `roll_no`): Identifies that student's **enrollment in a particular class for a particular academic year**.

In legacy ERPs, the class and roll number were stored directly on the student record. When a student was promoted, their class was overwritten, which accidentally re-parented old attendance marks and old invoices to the new class! Sunrise ERP prevents this by splitting them into two separate tables:

`students`

(One per student for life)

—1:Many—▶

`enrolments`

(One row per year attended)

`students.id` —PK to FK→ `enrolments.student_id`

Aspect	Student ID (<code>students</code>)	Enrollment ID (<code>enrolments</code>)
What it represents	The student as a permanent human being.	The student's seat in a specific class section for one school year.
Database Table	<code>students</code>	<code>enrolments</code> (spelled with a single 'l')
Primary Key	<code>students.id</code> (BigInt)	<code>enrolments.id</code> (BigInt)
Human Identifier	<code>admission_no</code> (e.g. ADM-2024-001)	<code>roll_no</code> (e.g. Roll #14 in Section 6-A)
When it is created	Created once when the student first joins the school.	Created every year when the student is admitted or promoted.
Does it ever change?	Never. Permanent throughout all school years.	Yes. A new enrollment record is created every academic year.
How many can a student have?	Exactly 1 record per student.	Multiple records (1 for each year the student spends in school).

Illustrative Multi-Year Example: Student Aarav Sharma

Notice how Aarav's `students.id` stays **12** forever, while he gets a new `enrolments.id` each year:

Academic Year	Student Record	Enrollment Record	Class & Section	Roll No	Status
2024-25	<code>students.id</code> = 12 (ADM-2024-001)	<code>enrolments.id</code> = 42	Class 5-A	Roll 12	promoted
2025-26	<code>students.id</code> = 12 (ADM-2024-001)	<code>enrolments.id</code> = 95	Class 6-A	Roll 14	promoted
2026-27	<code>students.id</code> = 12 (ADM-2024-001)	<code>enrolments.id</code> = 168	Class 7-A	Roll 15	active

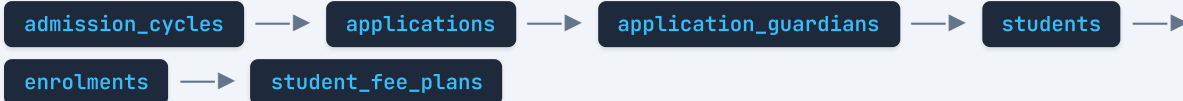
Which Downstream Tables Use Which ID? (Verified From Schema)

Downstream Tables Using <code>enrolment_id</code> (Year-Scoped Facts)	Downstream Tables Using <code>student_id</code> (Lifetime Facts)
• <code>attendance.enrolment_id</code> → Daily attendance belongs to that year's class. • <code>fee_invoices.enrolment_id</code> → Monthly invoices billed for that academic year. • <code>fee_payments.enrolment_id</code> → Payments collected against that year's dues. • <code>student_fee_plans.enrolment_id</code> → The class fee plan attached for that year. • <code>fee_concessions.enrolment_id</code> → Sibling/merit discount granted for that year. • <code>report_card_publications.enrolment_id</code> → Annual report card issued. • <code>transport_assignments.enrolment_id</code> → Bus seat allocated for that session. • <code>student_leave_requests.enrolment_id</code> → Pupil leave from active class.	• <code>student_guardian.student_id</code> → Permanent link to mother, father, or guardian. • <code>applications.student_id</code> → The original admission application that created the child. • <code>application_siblings.student_id</code> → Sibling discount tracking in admissions. • <code>grievances.student_id</code> → Permanent helpdesk complaint record. • <code>marks.student_id</code> → Subject exam marks awarded to the student. • <code>homework_submissions.student_id</code> → Homework turn-in records. • <code>message_recipients.student_id</code> → Broadcast SMS/push notifications.

2. Operational Features: Table-to-Table Data Flows

1. Student Admission & Conversion Pipeline

ADMISSIONS



admission_cycles → Stores the active academic admission intake year and dates.

applications → Stores applicant biodata, applied class, and application status.

application_guardians → Stores parent contacts and details submitted with the application.

students → Stores the permanent student record created upon successful conversion.

enrolments → Stores the child's active class section and roll number.

student_fee_plans → Links the new student enrollment to the standard class fee schedule.

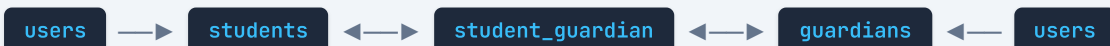
```
admission_cycles.id → applications.cycle_id
applications.id → application_guardians.application_id
users.id → students.user_id (Student login account)
students.id → enrolments.student_id
enrolments.id → student_fee_plans.enrolment_id
```

Why Data Moves: An applicant submits form details into **applications**. Once approved, clicking 'Convert' in the UI triggers an atomic transaction that promotes the applicant into a real **students** record, enrolls them in a class section via **enrolments**, and sets up their fee billing via **student_fee_plans**.

The admission process takes a prospective student from initial application to official enrollment. The applicant's personal data becomes permanent records in **students** and **guardians**, while their classroom placement is recorded in **enrolments**. The originating application is marked enrolled and links to the new student record.

2. Student & Guardian Family Master Linking

MASTER DATA



users → Stores login credentials, email/phone, and authentication state.

students → Stores lifetime child identity, date of birth, gender, and unique admission number.

guardians → Stores parent occupation, income, and emergency contact details.

student_guardian → Junction table connecting multiple children to their parents (M:N).

```
users.id → students.user_id | users.id → guardians.user_id
students.id → student_guardian.student_id
guardians.id → student_guardian.guardian_id
```

Why Data Moves: Both students and parents need login accounts (in **users**). Because parents often have multiple children in the same school (siblings), the **student_guardian** junction table connects them cleanly without duplicating parent profiles.

This structure represents family relationships in the school. A student row is created with a permanent admission number, and a guardian row is created for the parent. The junction table records who is the father, mother, or legal guardian, allowing sibling discounts and consolidated parent billing.

3. Academics, Class Sections & Timetable Slots

ACADEMICS

`academic_years` → `class_sections` → `class_subject_teacher` → `timetable_slots`

`academic_years` → Stores the academic calendar year (e.g. 2026-27).

`class_sections` → Stores classroom divisions (e.g. Class 10-A) and assigns the class teacher.

`class_subject_teacher` → Maps which teacher teaches which subject in that section.

`timetable_slots` → Stores the weekly timetable schedule (day of week, period, teacher, subject).

```
academic_years.id → class_sections.academic_year_id
class_sections.id → class_subject_teacher.class_section_id
class_sections.id → timetable_slots.class_section_id
employees.id → timetable_slots.teacher_id
```

Why Data Moves: Schools organize teaching by assigning faculty to subjects, and then scheduling those subjects into weekly timetable periods so teachers and students know where to be each period.

Academic years contain class sections. Each section has a designated class teacher and subject teachers. The timetable grid maps these teachers and subjects into concrete weekly periods, which later drives substitution conflict checking when teachers take leave.

4. Student Daily Attendance Roll Marking

ATTENDANCE

`class_sections` → `enrolments` → `attendance`

`class_sections` → Identifies the classroom being marked (e.g. Class 10-A).

`enrolments` → Provides the list of active students enrolled in that section this year.

`attendance` → Stores the daily roll-call mark (present, absent, late, leave) for that date.

```
class_sections.id → enrolments.class_section_id
enrolments.id → attendance.enrolment_id
```

Why Data Moves: When the class teacher opens the attendance screen, the system fetches the active student roster from enrolments. The teacher marks the roll, which inserts daily presence records into attendance.

Daily attendance points to enrolment_id rather than student_id because attendance belongs to the student's year in a specific class. If the student was absent in Class 5, that record remains tied to their Class 5 enrollment forever.

5. Fee Plan Setup & Monthly Invoice Billing

FEES & BILLING



fee_heads → Stores fee categories (e.g. Tuition, Computer Lab, Annual Sports).

fee_plans → Stores the standard fee structure for a class (e.g. Class 10 Annual Plan).

fee_plan_items → Stores the breakdown of amounts per fee head within that plan.

student_fee_plans → Attaches the plan to the student's enrollment.

fee_invoices → Stores the monthly invoice bill issued to the student.

fee_invoice_lines → Stores the individual charges on that invoice bill.

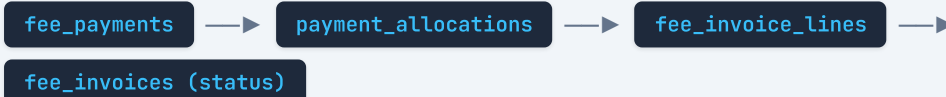
```
fee_heads.id → fee_plan_items.fee_head_id
fee_plans.id → fee_plan_items.fee_plan_id
enrolments.id → fee_invoices.enrolment_id
fee_invoices.id → fee_invoice_lines.invoice_id
```

Why Data Moves: At the start of the month, the billing engine reads the class fee plan, creates an invoice header in **fee_invoices**, and splits the total into granular charge lines in **fee_invoice_lines**.

Invoices are generated per enrollment. The system pulls all fee plan items, subtracts any active concessions from **fee_concessions**, and creates an official invoice. The invoice starts in unpaid status with individual line amounts ready for collection.

6. Counter Fee Collection & FIFO Line Allocation

FEE PAYMENTS



fee_payments → Stores the cashier receipt voucher (amount collected, payment mode, receipt no).

payment_allocations → Junction table linking the collected money to specific unpaid invoice lines.

fee_invoice_lines → The specific charges being settled (Tuition, Library, etc.).

fee_invoices → Header updated to paid or partial once lines are settled.

```
enrolments.id → fee_payments.enrolment_id
fee_payments.id → payment_allocations.payment_id
fee_invoice_lines.id → payment_allocations.invoice_line_id
```

Why Data Moves: When a parent pays money, the system creates a payment record and automatically allocates the money to the oldest unpaid invoice lines first (FIFO rule), updating the invoice status.

Money is never dumped as an unallocated lump sum. Every collected rupee is tracked through **payment_allocations** down to the exact invoice line it cleared. If a payment is reversed later (e.g. bounced cheque), a contra row in **fee_payments** reverses those exact allocations without deleting history.

7. Examinations, Marks Entry Grid & CBSE Grading

EXAMS

exams → exam_schedule → marks ← students ← grade_bands

exams → Stores the major examination event (e.g. Term 1 Midterms).

exam_schedule → Stores paper datesheet, max marks, and locking state (is_locked).

marks → Stores marks awarded to the student for that exam subject paper.

students → Identifies the student who took the exam.

grade_bands → Supplies CBSE letter grade (A1, A2, B1) and grade point for the percentage score.

```
exams.id → exam_schedule.exam_id
exams.id → marks.exam_id
students.id → marks.student_id
```

Why Data Moves: Teachers evaluate papers and submit scores on the marks entry grid. The system saves the score in marks, looks up the CBSE grade band, and locks the paper in exam_schedule to prevent tampering.

Exams link to subjects and datesheets. Marks awarded to students are checked against grade_bands to automatically compute CBSE grades (e.g. 91-100 = A1). When the exam controller locks the schedule, no further mark updates are allowed without an audited unlock reason.

8. Academic Session Rollover & Class Promotion

PROMOTION

enrolments (2025-26) → (marked 'promoted') enrolments (2026-27) → (inserted 'active')
→ audit_log

enrolments (Old Year) → Prior academic year record updated to status = 'promoted'.

enrolments (New Year) → Brand new record inserted for target academic year with new class section and roll number.

students → **NEVER MODIFIED.** Student ID and admission number remain identical.

audit_log → Records rollover execution metrics, timestamp, and operator signature.

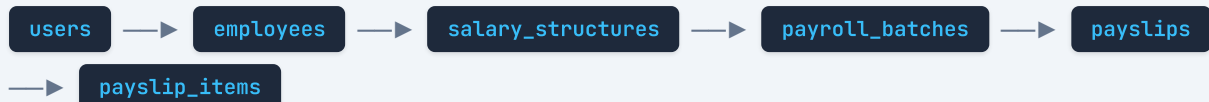
```
students.id —Identical on both→ enrolments.student_id
academic_years.id —Points to new year→ enrolments.academic_year_id
```

Why Data Moves: At the end of the school year, students advance to the next grade. The rollover wizard creates new enrollment records for the next session while marking old ones promoted, preserving 100% of past academic history.

Session rollover creates new rows instead of destroying old ones. A student moving from Class 5-A to Class 6-A keeps their permanent student ID, but receives a new enrollment record for 2026-27. This ensures old report cards and attendance records are never corrupted.

9. Staff Directory, Salary Structures & Monthly Payroll

HR & PAYROLL



users → Stores staff authentication login credentials.

employees → Stores employee code (e.g. TCH003), designation, department, and joining date.

salary_structures → Stores compensation base pay, allowances, and statutory rules.

payroll_batches → Header for monthly payroll calculation run (e.g. August 2026).

payslips → Individual employee salary slip (Gross, Total Deductions, Net Pay).

payslip_items → Itemized earnings (Basic, DA, HRA) and deductions (EPF, ESI, TDS).

```
users.id → employees.user_id
employees.id → salary_structures.employee_id
payroll_batches.id → payslips.batch_id
payslips.id → payslip_items.payslip_id
```

Why Data Moves: At month-end, the payroll officer calculates salaries. The engine reads each staff member's salary structure, generates individual payslips, calculates statutory deductions (EPF/ESI), and exports bank disbursements.

Staff records live in **employees**. Each employee has an assigned salary structure. When the monthly payroll batch is calculated, the system creates a payslip for each employee with itemized lines for basic pay, allowances, and EPF/TDS deductions.

10. Teacher Mobile Leave & 100% Substitution Coverage Gate

LEAVE & SUBSTITUTION



employees → The teacher applying for leave.

staff_leave_requests → Stores the leave application (dates, reason, status).

timetable_slots → Identifies which teaching periods become vacant on those dates.

substitutions → Temporary assignments linking free teachers to vacant timetable slots.

in_app_notifications → Push alerts sent to substitute teachers informing them of duties.

```
employees.id → staff_leave_requests.employee_id
staff_leave_requests.id → substitutions.leave_request_id
timetable_slots.id → substitutions.timetable_slot_id
employees.id → substitutions.substitute_teacher_id
```

Why Data Moves: A teacher applies for leave via mobile. The system finds their timetable slots for that day. The admin must assign substitute teachers to 100% of vacant slots before the 'Approve' button unlocks.

When a teacher takes leave, their classroom periods must not be left unattended. The system detects conflicts in **timetable_slots**, allows the admin to assign free teachers in **substitutions**, and sends mobile alerts. Master timetable slots are never modified; substitutions expire automatically.

11. Teacher Recruitment Pipeline & 1-Click Onboarding

RECRUITMENT

`candidates` → `candidate_offers` → `users` → `employees` → `user_roles`

`candidates` → Stores job applicant bio-data, resume, position applied, and hiring stage.

`candidate_offers` → Stores the formal job offer parameters (salary, designation, joining date).

`users` → User account created automatically when candidate is onboarded.

`employees` → Permanent faculty record created with new staff code (e.g. TCH014).

`user_roles` → Assigns the 'Teacher' system role to the newly created user.

`candidates.id` → `candidate_offers.candidate_id`

`users.id` → `employees.user_id`

`candidates.employee_id` → `employees.id` (Points to created staff)

Why Data Moves: An applicant advances through interview stages to an accepted job offer. Clicking 'Onboard' automatically provisions them as an active employee and user without retyping information.

The recruitment module tracks teaching applicants from application to hiring. When a candidate accepts their offer, 1-click onboarding creates their system login in `users` and adds them to `employees`, immediately making them available for timetable assignment.

12. Consumable Stock Inventory & Requisition Workflow

INVENTORY

`stock_categories` → `stock_items` → `stock_requests` →

`stock_items` (quantity decremented)

`stock_categories` → Groups items into departments (e.g. Science Lab, Stationery, Sports).

`stock_items` → Stores item name, current quantity, minimum safe threshold, and unit.

`stock_requests` → Stores staff requests for items and approval state.

`stock_categories.id` → `stock_items.category_id`

`stock_items.id` → `stock_requests.item_id`

`employees.id` → `stock_requests.requested_by_id`

Why Data Moves: Staff request consumables (e.g. whiteboard markers, test tubes). When the storekeeper approves the request, the system deducts the quantity from `stock_items` and flags an alert if stock drops below minimum threshold.

The inventory catalog tracks school physical assets and consumables. Staff requisitions in `stock_requests` reduce physical stock upon approval. If remaining quantity falls below `min_quantity`, an amber warning badge appears on the admin dashboard.

13. Parent / Staff Grievance Redressal & Helpdesk

GRIEVANCES

`users / students` → `grievances` → `grievance_replies` →
`grievances (status='resolved')`

`grievances` → Stores the complaint ticket (ticket number, title, category, assigned staff, status).

`grievance_replies` → Stores threaded conversation replies between parent and school staff.

`employees` → The staff member assigned to resolve the grievance.

```
students.id → grievances.student_id (Optional student link)
employees.id → grievances.assigned_to_id
grievances.id → grievance_replies.grievance_id
```

Why Data Moves: A parent files a complaint on mobile. The admin assigns the ticket to a staff member in `grievances`. Staff and parent exchange messages in `grievance_replies` until the ticket is resolved.

The grievance system manages school complaints and inquiries. Each ticket starts in open status, receives threaded responses in `grievance_replies`, and transitions to resolved once administrative action is taken.

14. Fleet Transport Routes, Stops & Student Bus Assignments

TRANSPORT

`routes` → `route_stops` ↔ `vehicles` → `transport_assignments` ← `enrolments`

`routes` → Stores the bus route name, code, and operational status.

`route_stops` → Stores individual pickup/drop locations along the route and stop timing.

`vehicles` → Stores bus vehicle details, registration, seating capacity, and driver details.

`transport_assignments` → Links an enrolled student to a specific bus stop and route.

```
routes.id → route_stops.route_id
enrolments.id → transport_assignments.enrolment_id
route_stops.id → transport_assignments.route_stop_id
```

Why Data Moves: When parents opt for school bus service, the student's active enrollment is linked to a designated route stop in `transport_assignments`, ensuring the student appears on driver passenger manifests.

The transport module configures routes and designated stops. Bus seats are allocated per academic session through `enrolment_id`. Vehicle paper expiry dates are monitored in `vehicles` to ensure regulatory compliance.

15. Authentication, System Roles & Granular Permissions

SECURITY & RBAC

`users` → `user_roles` → `roles` → `role_permissions` → `permissions`

`users` → Stores email, password hash, and active status.

`user_roles` → Maps which user holds which role (e.g. Admin, Teacher, Receptionist).

`roles` → Catalog of system roles defined for the school.

`role_permissions` → Junction mapping permissions to each role.

`permissions` → Master catalog of granular action strings (e.g. `fees.invoice.read`).

```
users.id → user_roles.user_id
roles.id → user_roles.role_id
roles.id → role_permissions.role_id
permissions.id → role_permissions.permission_id
```

Why Data Moves: When a user logs in, the backend traverses this chain to collect all permission codes granted to their assigned roles, embedding them into their access token to gate screens and API routes.

Security in Sunrise ERP is completely role-based. Users are assigned roles in `user_roles`, and roles are linked to permissions in `role_permissions`. The frontend UI hides unauthorized menus, and the backend blocks unauthorized API endpoints with HTTP 403.

3. Important IDs Glossary

Identifier Name	Database Column & Type	Simple Operational Meaning
School ID	school_id (BigInt)	Identifies the school tenant. Present on 90 tables to keep each customer school's data completely isolated.
User ID	users.id (BigInt)	Identifies a human login account (Admin, Teacher, Student, Parent, or Receptionist).
Student ID	students.id (BigInt)	The internal primary key for the student's permanent lifetime record.
Admission Number	students.admission_no (String)	The permanent human-readable student code (e.g. ADM-2024-001) printed on ID cards. Never changes.
Enrollment ID	enrolments.id (BigInt)	The primary key identifying a student's enrollment in a specific class for a specific academic year.
Roll Number	enrolments.roll_no (Integer)	The student's roll number within their class section for that year (e.g. Roll #12 in 5-A). Unique within the section.
Employee ID	employees.id (BigInt)	The internal database primary key for a staff member (teaching or non-teaching).
Employee Code	employees.employee_code (String)	The permanent staff number (e.g. TCH001, TCH014) used in timetable and payroll records.
Invoice ID	fee_invoices.id (BigInt)	The unique identifier for a monthly fee bill issued to a student enrollment.
Payment ID	fee_payments.id (BigInt)	The unique identifier for an immutable fee payment receipt voucher collected at the counter.
Exam ID	exams.id (BigInt)	Identifies a major examination cycle (e.g. Term 1 Exams).

4. Big-Picture ERP Database Flow

```
===== SUNRISE ERP: BIG-  
PICTURE DATABASE FLOW  
=====
```

[admission_cycles] | ▼ [applications] | (Conversion) ▼

[student_guardian] ← [students] (Permanent Identity) | | ▼▼ [users] → [guardians] ←

| | [enrolments] (Annual Membership) | ↳ [staff_leave_requests] | | | ↳ [attendance] | | ▼ | | |

[substitutions] ← [timetable_slots] ↳ [fee_invoices] → [fee_invoice_lines] | | | ▲ | ↳

[staff_attendance] | | | | | ↳ [fee_payments] → [payment_allocations] | ↳ [payslips] →

[payslip_items] | | | ↳ [transport_assignments] ↳ [user_roles] → [roles] → [permissions] | ↳

[report_card_publications] ▲ | (Marks & Grading) [exams] → [exam_schedule] → [marks] ← [students]

```
=====
```

5. The Final Student Data Journey

Here is the complete journey of a child's data through the database from their first inquiry to graduation:

```
1. INTAKE: enquiries → applications → application_guardians / documents | ▼ (Conversion) 2. PERMANENT
IDENTITY: users → students (admission_no: 'ADM-2024-001') ← student_guardian → guardians | ▼
(Enrolled into Class 5-A for 2024-25) 3. ANNUAL ACADEMIC SESSION (Year 1): enrolments (id: 42,
class_section_id: 5-A, roll_no: 12, status: 'active') | | → attendance (Daily Roll-Call Marks for Class
5-A) | | → fee_invoices → fee_invoice_lines (Tuition, Lab) ← payment_allocations ← fee_payments |
| → marks (Exam Scores Awarded to student_id) → CBSE Grade Bands (A1, A2) | | → transport_assignments
(Bus Route Stop Allocated) | ▼ (March: Academic Session Rollover) 4. PROMOTION TO NEXT GRADE (Year 2): •
enrolments (id: 42) → Updated to status: 'promoted' (Past history untouched!) • enrolments (id: 95) →
INSERTED for Class 6-A in 2025-26 with roll_no: 14, status: 'active' • students (id: 12) → UNTOUCHED!
Lifetime identity remains 'ADM-2024-001'. | ▼ 5. REPEAT CYCLE FOR EVERY SUBSEQUENT GRADE UNTIL GRADUATION!
```

Summary Takeaway:

In Sunrise ERP, **data never gets overwritten or destroyed**. Permanent identities live in students and employees. Annual facts live in enrolments and academic_years. Transactions live in attendance, fee_invoices, fee_payments, marks, and payslips. Destructive actions are logged in audit_log.